

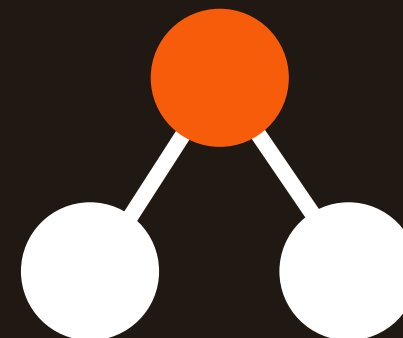
|TIMUS 2160 Мета-условие

ПОДСЧЁТ ЭКВИВАЛЕНТНЫХ ШАБЛОНОВ ПЕРЕСТАНОВОК

ЧЕРЕЗ **ДЕКАРТОВО ДЕРЕВО** И **КОМБИНАТОРИКУ**

Стебенов Иван Андреевич

КРБО-12-24



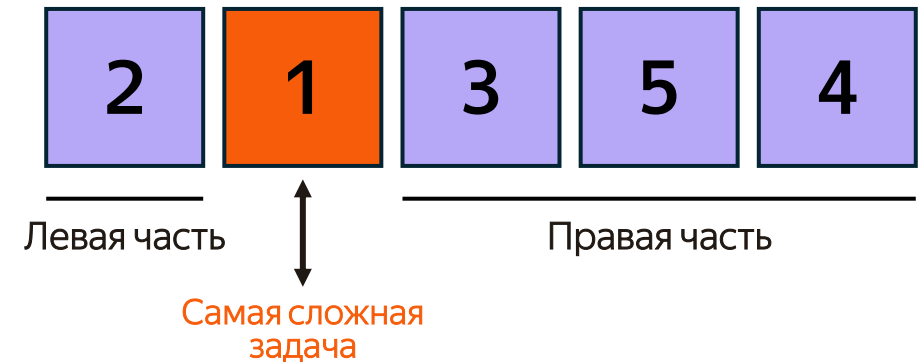
ЖЮРИ РАССТАВЛЯЕТ ЗАДАЧИ В КОНТЕСТЕ

В комплекте — **N задач**, отсортированных по сложности (1 — самая сложная).

Чтобы расставить их по местам, берётся **шаблон** — перестановка чисел от 1 до N.

Старый шаблон **надоел**. Нужно посчитать, сколько других перестановок ему **эквивалентны** — это и будет ответ.

ШАБЛОН



Алгоритм определения шаблона: найти минимум, разбить шаблон на левую и правую часть, рекурсивно повторить для каждой части.

ПОСТАНОВКА

КОГДА ДВА ШАБЛОНА ЭКВИВАЛЕНТНЫ?

Два шаблона **эквивалентны**, если оба шаблона пустые, либо у них совпадает позиция минимума, и их левые и правые подшаблоны эквивалентны

Какие из этих шаблонов эквивалентны (2, 1, 3)?

(2, 1, 3)

2

1

3

ЭТАЛОН

(2, 1, 3)

2

1

3

ЭКВИВАЛЕНТЕН

(1, 2, 3)

1

2

3

НЕ ЭКВИВАЛЕНТЕН

ПОСТАНОВКА

ФОРМАЛЬНАЯ ПОСТАНОВКА ЗАДАЧИ

Вход

N — количество задач, $1 \leq N \leq 10^5$.

$p_1, p_2, \dots, p_n$ — перестановка чисел от 1 до N.

Найти

Количество перестановок, **эквивалентных** данной, по модулю $10^9 + 7$.

Ограничения

Время: **≤ 2 секунд**

Память: **≤ 256 МБ**

ПРИМЕР 1

ВХОД

3
2 1 3

ОТВЕТ

2

ПРИМЕР 2

ВХОД

5
3 1 4 2 5

ОТВЕТ

8

ПЕРЕБОР «В ЛОБ» НЕ ПОДОЙДЁТ

НАИВНЫЙ АЛГОРИТМ

1. Перебрать все $N!$ перестановок
2. Для каждой проверить **эквивалентность**
3. Сосчитать подходящие

Сложность: $O(N \cdot N!)$

...а нам нужно работать при N до 10^5

ПРИ $N = 20$

$N!$

$\approx 2.4 \cdot 10^{18}$

операций

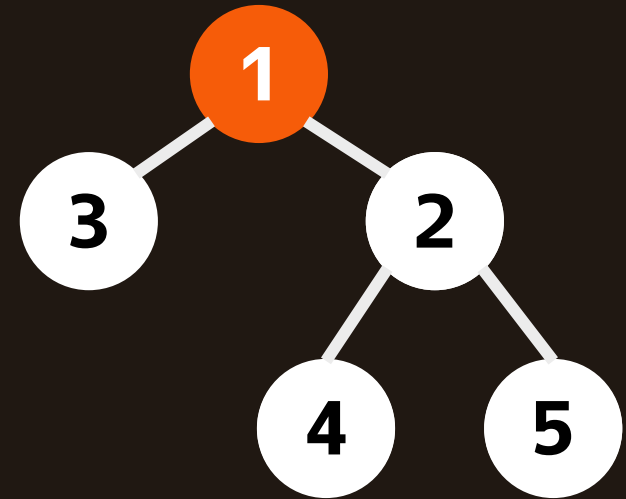
даже близко не уложиться в 2 секунды...

Шаблон — это

декартово дерево



$(3, 1, 4, 2, 5)$



Корень — минимум массива. Левое и правое поддеревья строится рекурсивно из элементов находящихся слева и справа от корня.

ЧТО ТАКОЕ ДЕКАРТОВО ДЕРЕВО

Это **бинарное дерево**, в котором сочетаются два свойства одновременно.

СВОЙСТВО 1

BST по позициям

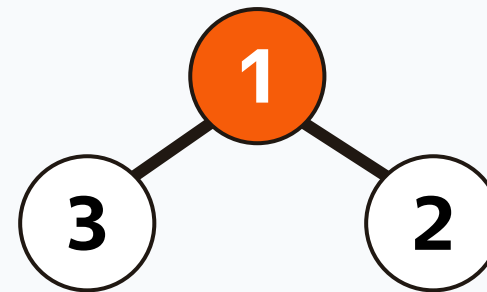
Слева направо — исходный массив.



СВОЙСТВО 2

min-heap по значениям

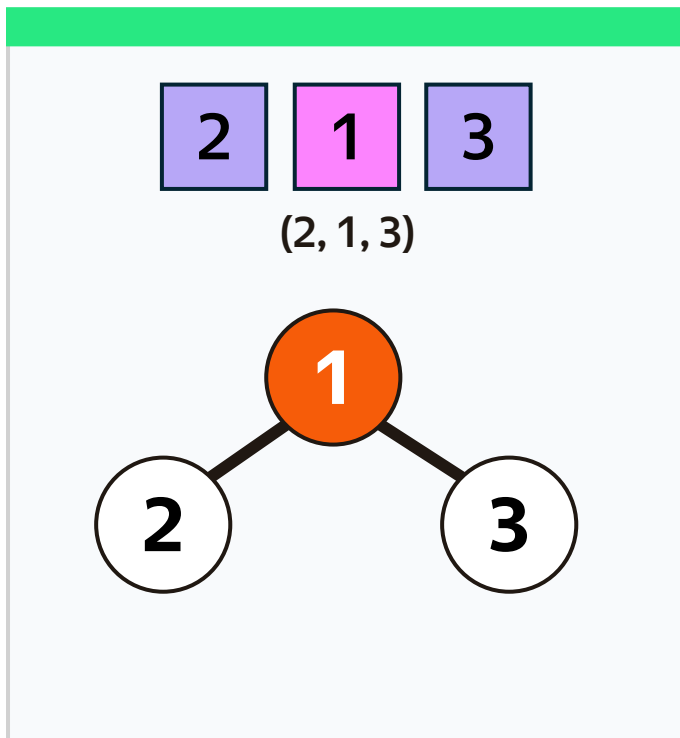
Сверху вниз — значения только растут.
Корень — глобальный минимум.



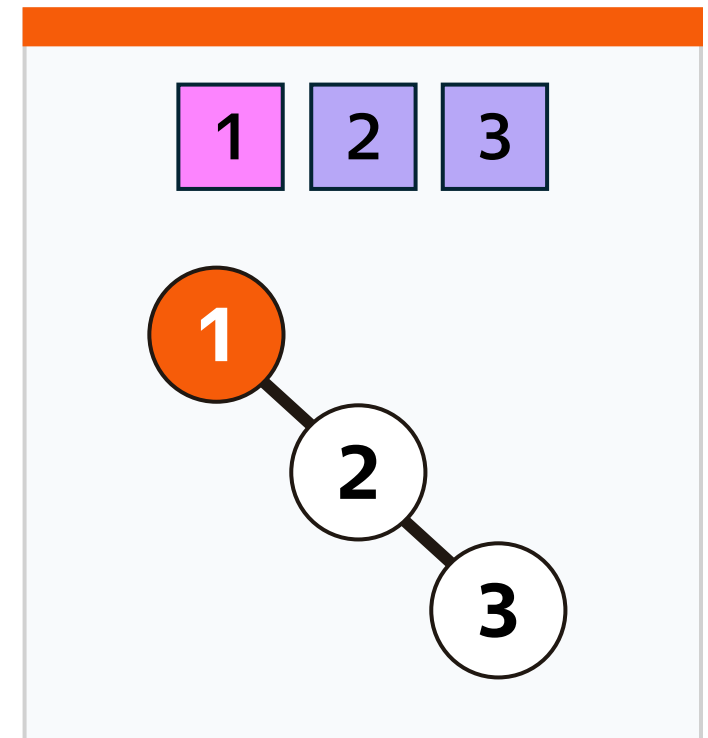
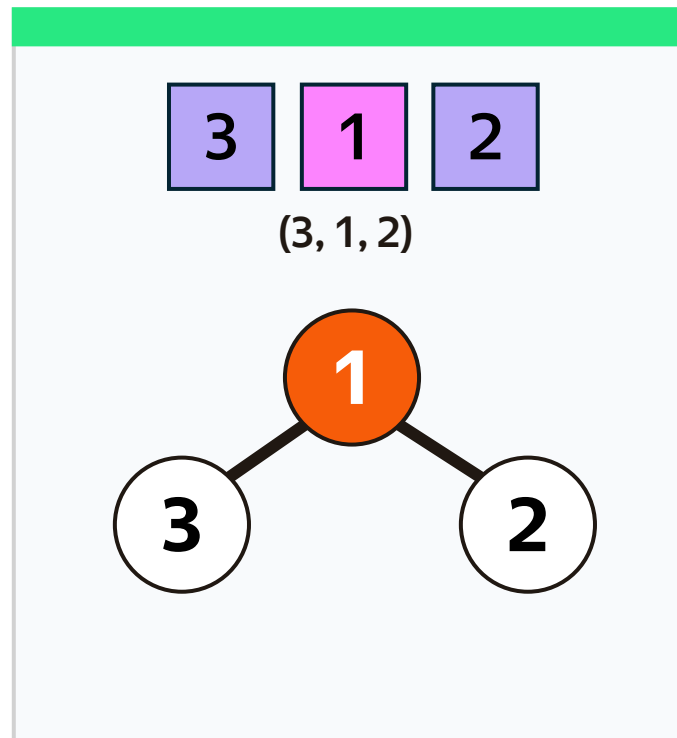
Применяется в RMQ-запросах, парсерах выражений, многих сортировках. По массиву с уникальными значениями строится однозначно.

Эквивалентность $\Leftrightarrow$ изоморфизм деревьев

Два шаблона **эквивалентны** тогда и только тогда, когда у их декартовых деревьев совпадает форма (без учёта значений).



ОДИНАКОВАЯ ФОРМА $\rightarrow$ ЭКВИВАЛЕНТНЫ



ДРУГАЯ ФОРМА $\rightarrow$ НЕ ЭКВИВАЛЕНТЕН

РЕШЕНИЕ

Сколько перестановок дает
исходное дерево?

$$\prod (C_{L+R}^L) \mod 10^9 + 7$$

$\prod$ - произведение по всем вершинам

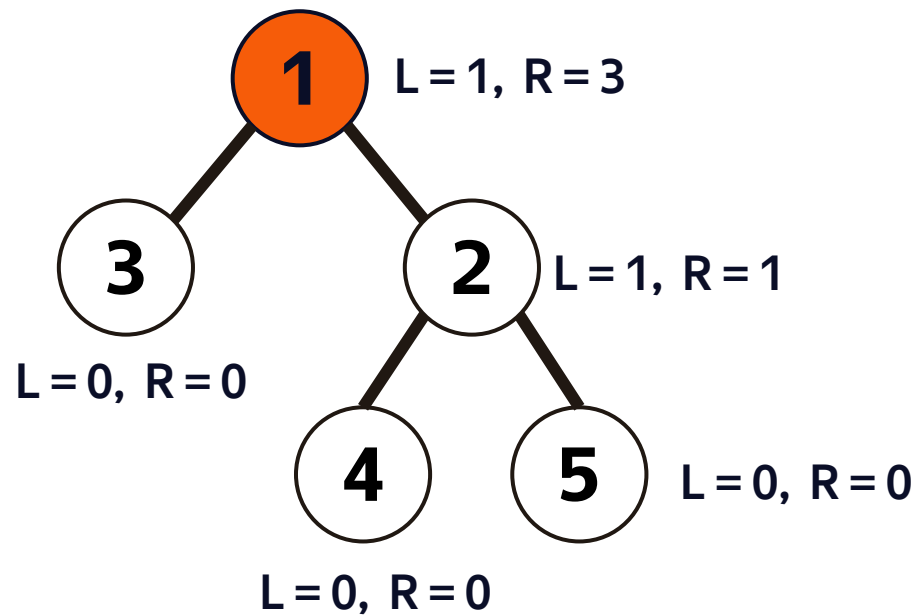
L из них уходят в левое поддереву

$L + R$ общее число потомков
вершины

$\mod 10^9 + 7$ ответ берут по модулю
т.к. числа огромные

РЕШЕНИЕ

Считаем для шаблона (3, 1, 4, 2, 5)



РАСЧЕТ ПО УЗЛАМ

УЗЕЛ 1 $C(4, 1) = 4$

УЗЕЛ 3 $C(0, 0) = 1$

УЗЕЛ 2 $C(2, 1) = 2$

УЗЕЛ 4 $C(0, 0) = 1$

УЗЕЛ 5 $C(0, 0) = 1$

Произведение = $4 \cdot 1 \cdot 2 \cdot 1 \cdot 1 = 8$

ОТВЕТ: **8**

ПОСТРОЕНИЕ ДЕРЕВА ЗА $O(N)$

ИДЕЯ

Идём по массиву слева направо. В **стеке** держим узлы по возрастанию значений снизу вверх.

Для каждого нового x :

1. пока вершина стека $> x$ — выталкиваем; последний вытолкнутый становится левым потомком x
2. если стек не пуст — x становится правым потомком новой вершины стека
3. кладём x в стек

Каждый элемент входит в стек и выходит ровно один раз → $O(N)$

PYTHON

```
stack = []
for x in arr:
    node = Node(x)
    last = None
    while stack and stack.top > x:
        last = stack.pop()
    node.left = last
    if stack:
        stack.top.right = node
    stack.push(node)
return stack[0] # корень
```

ПО ШАГАМ

СТРОИМ ДЕКАРТОВО ДЕРЕВО ДЛЯ МАССИВА (3, 1, 4, 2, 5)

X=3 стек пустой → кладём 3

СТЕК

3 — top



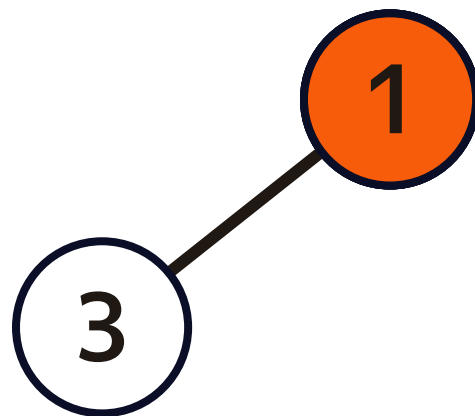
ПО ШАГАМ

СТРОИМ **ДЕКАРТОВО ДЕРЕВО** ДЛЯ МАССИВА **(3, 1, 4, 2, 5)**

X=1 $3 > 1 \rightarrow 3$ уходит влево от 1

СТЕК

1 — top



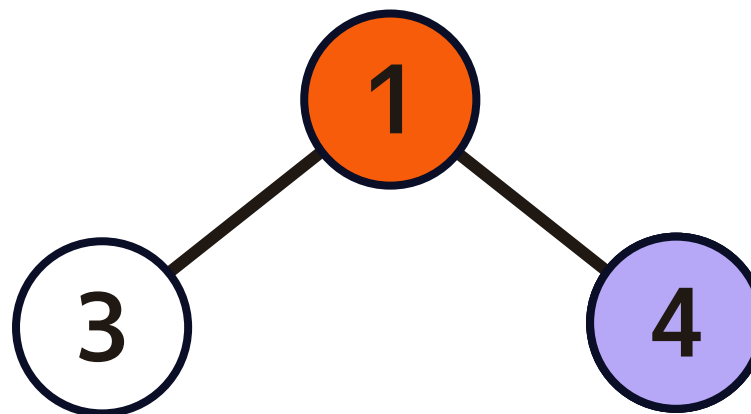
ПО ШАГАМ

СТРОИМ ДЕКАРТОВО ДЕРЕВО ДЛЯ МАССИВА (3, 1, 4, 2, 5)

X=4 $1 < 4 \rightarrow 4$ правый потомок 1

СТЕК

1 — top

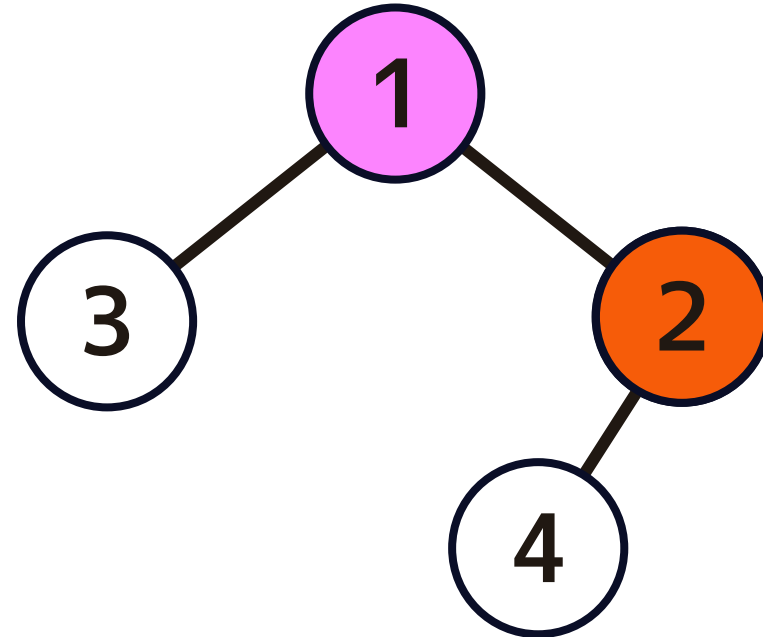
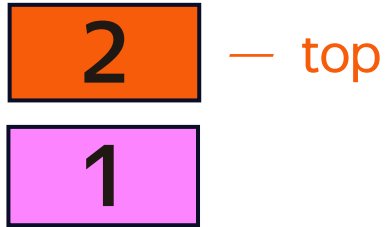


ПО ШАГАМ

СТРОИМ **ДЕКАРТОВО ДЕРЕВО** ДЛЯ МАССИВА **(3, 1, 4, 2, 5)**

X=2 $4 > 2 \rightarrow$ pop, 4 уходит под 2

СТЕК

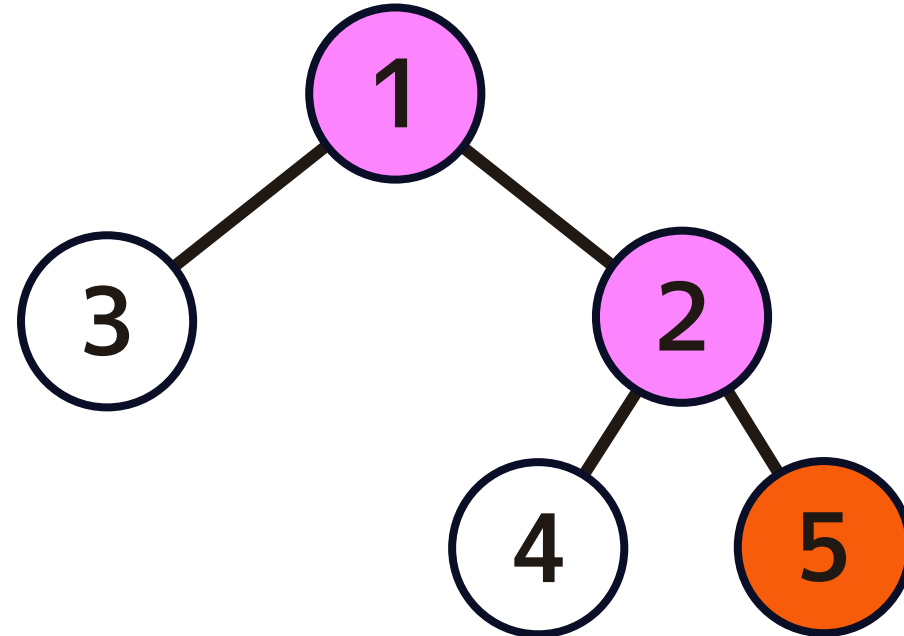
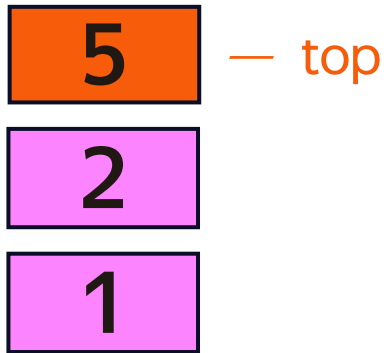


ПО ШАГАМ

СТРОИМ **ДЕКАРТОВО ДЕРЕВО** ДЛЯ МАССИВА **(3, 1, 4, 2, 5)**

X=5 $2 < 5 \rightarrow 5$ правый потомок 2

СТЕК

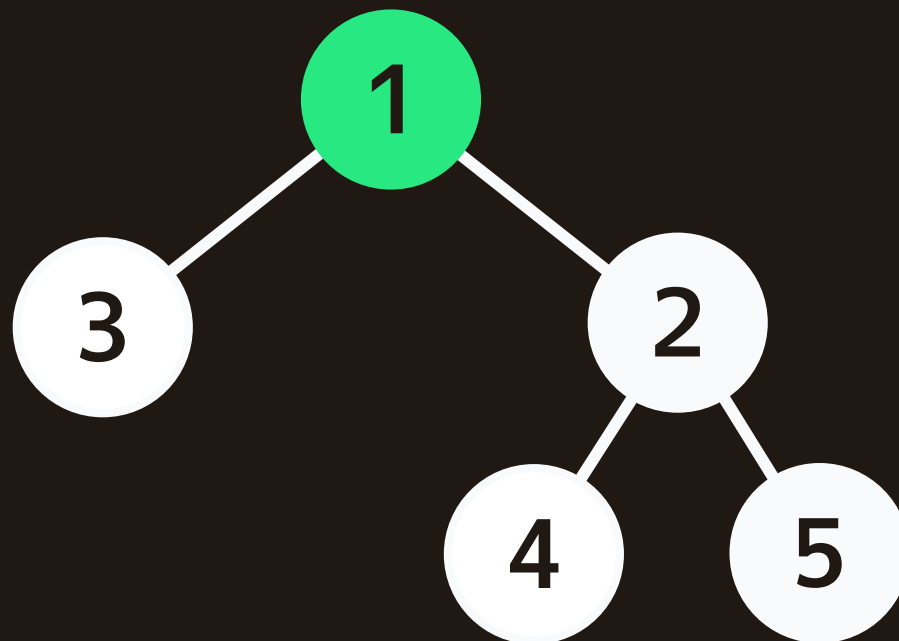
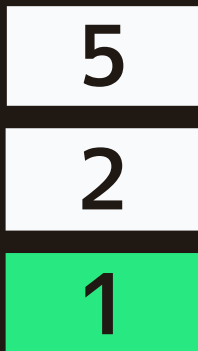


ИТОГ

КОРЕНЬ = СТЕК[0]

Дерево готово, теперь считаем $\prod(C_{L+R}^L)$

СТЕК



РЕАЛИЗАЦИЯ

$C(n, k)$ по модулю простого числа

ПРОБЛЕМА

$$C_n^k = \frac{n!}{(n-k)! \cdot k!}$$

По модулю обычного деления нет — нужен **обратный элемент**.

МАЛАЯ ТЕОРЕМА ФЕРМА

Если p простое и a не делится на p :

$$a^{-1} \equiv a^{(p-2)} \pmod{p}$$

→ считаем за $O(\log p)$ быстрым возведением в степень.

ОПТИМИЗАЦИЯ

Один раз предвычисляем массивы:

$$\text{fac}[i] = i! \bmod p$$

$$\text{invfac}[i] = (i!)^{-1} \bmod p$$

Используем рекуррентность $(k-1)!^{-1} = k!^{-1} \cdot k$
— один row на старте, дальше всё за $O(N)$.

Тогда любой

$$C(n, k) = \text{fac}[n] \cdot \text{invfac}[k] \cdot \text{invfac}[n-k]$$

Выполняется за $O(1)$ на запрос.

Программа состоит из четырёх классов

Node

узел декартова дерева

- value, left, right
- size_val — размер поддеревы

Stack

стек на списке

- push / pop / top / is_empty

Combinatorics

биномиальные коэффициенты

- fac[], invfac[] — предвычисление
- $C(n, k)$ за $O(1)$

CartesianTree

оркестратор алгоритма

- build() — стек, $O(N)$
- count() — итерация + $\prod C(L+R, L)$

Главные грабли: рекурсия по дереву

ПРОБЛЕМА

На отсортированном входе дерево вырождается в линейную цепочку.

Глубина рекурсии = N . При $N = 10^5$ это переполнение стека вызовов и аварийное завершение.

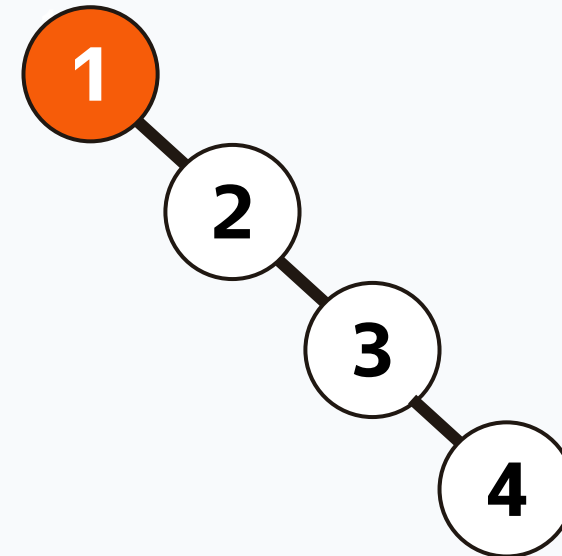
РЕШЕНИЕ

Все обходы дерева — **итеративные** через явный список-стек.

Глубина теперь ограничена только размером кучи, а не системного стека.

ВЫРОЖДЕННОЕ ДЕРЕВО

вход (1, 2, 3, 4)



глубина = N

Итоговая сложность и проверка

ПОСТРОЕНИЕ ДЕРЕВА

 $O(N)$ *МОНОТОННЫЙ СТЕК*

ВЫЧИСЛЕНИЕ ОДНОГО С

 $O(1)$ *после предвычислений*

ИТОГ

 $O(N)$ *линейно, обходимся одним проходом*

ID	Дата	Автор	Задача	Язык	Результат проверки	№ теста	Время работы	Выделено памяти
11174360	19:53:12 23 мар 2026	IvanStebenev	2160. Мета-условие	Python 3.12 x64	Accepted		1.14	39 364 КБ

Спасибо за внимание!

